08 — Wallet Key Hardening Plan (KMS/HSM-backed signer)

> The single highest-leverage security item. Today, `WALLET_DERIVATION_SECRET` (and `ENCRYPTION_KEY`) are environment variables read by the application servers, and from the derivation secret + a phone number **every** user's private key can be reconstructed. Compromise of that one value = total loss of all user funds. This plan removes that single point of catastrophic failure.

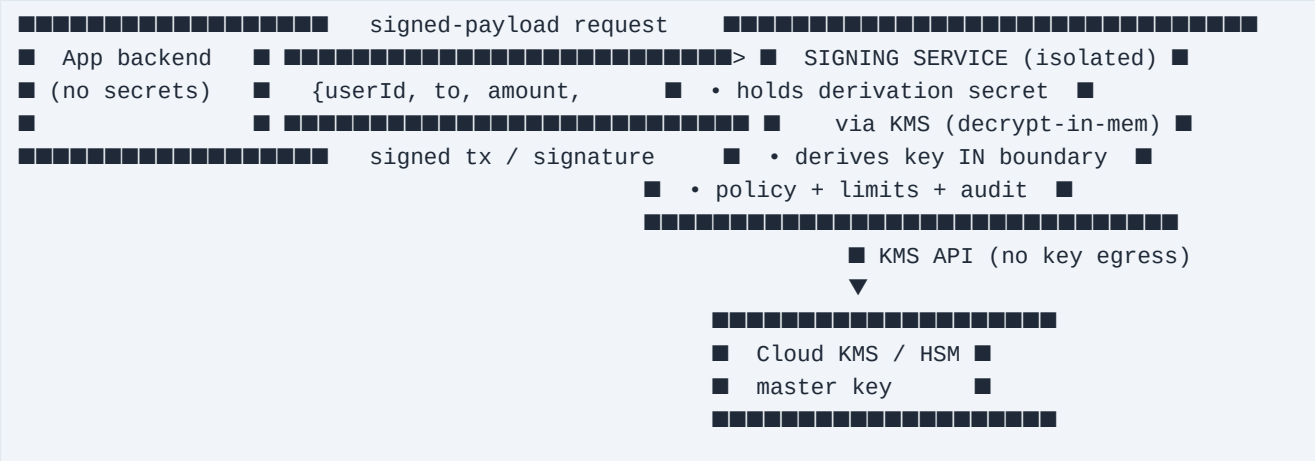
1. Threat being addressed

- **Master-secret exfiltration:** anyone who reads `WALLET_DERIVATION_SECRET` (leaked env, compromised host, malicious insider, log/backup exposure) can derive and drain every wallet.
- **Blast radius:** 100% of user funds, instantly, irreversibly.
- **Current mitigations:** env-var access control + wallet/fee separation — necessary but not sufficient for scale.

2. Objective (end state)

1. The derivation secret lives **only inside a hardened boundary** (cloud KMS/HSM or an isolated signing service), never in app memory or app env.
2. Application servers **never see raw user private keys** — they request "sign this specific transfer for user X" and receive a signed transaction.
3. Every signing request is **authenticated, authorised, rate-limited, and audit-logged**, with anomaly alerts.

3. Target architecture



- The signing service is a **separate deployment** with its own minimal network surface (no public ingress; only the backend can call it, over mTLS / signed internal auth).
- The master secret is stored in **KMS** (e.g. AWS KMS, GCP KMS, Cloudflare, or a dedicated HSM). The service decrypts it into memory at boot (or, better, performs derivation/signing via KMS so the raw key never leaves KMS).

4. Phased rollout (low-risk, incremental)

Phase 0 — Immediate mitigations (days) - Restrict who/what can read `WALLET_DERIVATION_SECRET` / `ENCRYPTION_KEY` to the absolute minimum; remove from any non-essential service/log. - Add an **alert** on access to / change of these secrets. - Confirm secrets are **not** in backups, error reports (Sentry scrubbing), or client bundles. - Document an incident runbook for suspected exposure (freeze + sweep).

Phase 1 — Extract an internal signing service (1–2 weeks) - Move all code paths that call `deriveKeypairFromPhone` / `getUserKeypair` behind a single internal service interface (`POST /sign`). - The backend stops importing the secret; it calls the signer over an authenticated internal channel (mTLS or signed service token). - Signer enforces **policy** (per-user/tier limits re-checked, allow-listed operations) and **audit-logs** every signature.

Phase 1 is implemented (code): the app now sources its sensitive secrets through a single provider (`services/secrets.ts`) with an env fallback, so the master secret can be moved out of plain platform env into a managed store **without any code change**. To activate **Infisical** (free tier): 1. Create an Infisical project; in its `prod` environment add `WALLET_DERIVATION_SECRET`, `ENCRYPTION_KEY`, `JWT_SECRET`, `JWT_REFRESH_SECRET` (and any others you want vaulted). 2. Create a **Machine Identity (Universal Auth)** with read access to the project. 3. Set these env vars on the backend (Railway): `INFISICAL_CLIENT_ID`, `INFISICAL_CLIENT_SECRET`, `INFISICAL_PROJECT_ID`, `INFISICAL_ENV=prod` (and `INFISICAL_API_URL` only if self-hosting). 4. Deploy; confirm the boot log shows `[secrets] loaded N secret(s) from managed store (prod)`. 5. **Then remove** the migrated secrets from plain Railway env — the app now reads them from Infisical, with access logged there.

> [!] Do **not** remove `WALLET_DERIVATION_SECRET` / `ENCRYPTION_KEY` from env until you've confirmed Infisical is serving them (the boot log + a working login/transfer), or wallets won't derive.

Phase 2 — KMS/HSM-backed key custody (2–4 weeks) - Store the master secret in **KMS**; the signer obtains it only via KMS decrypt at boot, or derives/signs through KMS so the raw key never egresses. - Lock down the signer host: no shell, minimal image, network egress allow-list, read-only FS. - Add **rate limits + anomaly detection** on signing volume/patterns.

Phase 3 — Optional advanced custody (later) - Evaluate **threshold/MPC signing** (no single machine ever holds a full key) or a managed qualified-custody provider for the highest-value treasury. - Re-assess whether a partial **non-custodial** option is desirable for advanced users (trade-off: loses phone-only recovery).

5. The rotation problem (important)

Because addresses are derived from the master secret, **rotating it changes every user's address** — you cannot rotate in place without migrating funds. Therefore: - Treat the secret as **long-lived and protected by access control**, not by frequent rotation. - If rotation is ever required (suspected compromise), the procedure is a **sweep migration**: derive old keys inside the boundary, generate new wallets under a new secret, and move balances on-chain in a controlled batch. - Keep this sweep runbook written and tested (table-top) in advance.

6. Controls checklist (target)

Control	Target
Master secret in KMS/HSM, not app env	[Yes]
App servers never hold raw user keys	[Yes]
Signing isolated behind authenticated internal API	[Yes]
Per-request policy + tier-limit re-check	[Yes]
Full signing audit log + anomaly alerts	[Yes]
Secret access alerting; excluded from logs/backups/bundles	[Yes]
Documented + tested key-compromise sweep runbook	[Yes]
Independent security review / pen-test of the signer	[Yes]

7. Why this matters for partner/regulator review

A reviewer **will** notice "one master key controls all funds." Presenting this plan — with Phase 0 already done — turns a red flag into evidence of mature custody-risk management, and is the right thing to do regardless of optics.